ARCHON

Verifiable DevTools for Mantle

A technical whitepaper for Archon's audit engine, Mantle gas optimization system, ERC-8004 identity model, on-chain proof architecture, and public challenge ledger.

V2.9 public whitepaper

Built for teams that need audit artifacts, gas reports, CI comments, leaderboard records, proof metadata, and challengeable public validation records that can be traced beyond a static PDF.

Generated 2026-06-08 · archonaudit.xyz

Archon whitepaper

Executive summary

Archon is a verifiable security and gas-optimization platform for Mantle builders. It turns Solidity source into structured audit reports, Mantle-aware gas reports, public proof artifacts, and challengeable validation records. The product is intentionally pragmatic: it does not claim to replace expert review, and it does not treat AI output as unquestionable truth. Instead, it preserves provenance, separates measured values from estimates, and makes reports easier to inspect, export, integrate, and verify.

This paper argues that the next generation of crypto DevTools will be judged by evidence quality. A useful tool must show what source was analyzed, what assumptions were used, what was measured, what was estimated, which proof belongs to the artifact, and how another developer can challenge or reproduce the result. Archon's architecture — queued workers, deterministic hashing, source-bundle compilation, Fumadocs documentation, public API, GitHub Action, gas leaderboard, proof registry integration, and public challenge ledger — is built around that evidence trail.

Contents

Abstract

Smart-contract security has become a coordination problem as much as a code-quality problem. Teams ship increasingly complex protocols, integrate across chains, and depend on external reviewers, internal checklists, static analyzers, and PDF reports that are difficult to verify after the fact. A report may look authoritative, but the reader often cannot answer simple operational questions: what exact source was analyzed, what assumptions were used, which findings were deterministic, which were model-assisted, which proof belongs to this report, and whether the claimed optimization actually reduced cost under the target chain's fee model.

Archon is a Mantle-native DevTools platform designed to make smart-contract audits and gas optimization measurable, reproducible, and publicly verifiable. The product turns a repository, uploaded Solidity source, pasted contract, or verified-source input into three artifacts: an explainable audit report, a Mantle-aware gas optimization report, and an on-chain proof record tied to Archon's agent identity. The system is intentionally practical: it combines deterministic static analysis, Mantle-specific rules, reproducible source hashing, AI-assisted explanation, CI workflows, a public gas leaderboard, and proof metadata that can be anchored on-chain.

undefined

This whitepaper describes Archon's thesis: the next generation of developer security tooling will not be judged only by how convincing its UI looks, but by whether its outputs can be traced, challenged, reproduced, and integrated into the developer workflow. Archon is not a replacement for expert human audits, formal verification, or protocol-specific review. It is a verifiable DevTools layer that gives builders faster feedback, gives teams stronger evidence trails, and gives ecosystems a way to rank and reason about security and cost improvements using real artifacts rather than unverifiable claims.

The problem

Audits are still distributed as unverifiable PDFs

The dominant audit artifact is still a static report. That report may contain useful findings, but it usually behaves like a final presentation rather than a verifiable record. A reader is expected to trust the audit brand, the reviewer, or the client announcement. In practice, teams and ecosystems need more granular evidence.

A useful audit artifact should answer:

- Which source tree, commit, verified contract, or uploaded file was analyzed?
- Which compiler assumptions, network assumptions, and protocol assumptions were in scope?
- Which findings came from deterministic tools, which came from rule engines, and which were model-assisted explanations?
- Can a finding be mapped back to an exact file, line range, category, severity, and recommended fix?
- Can the report be hashed, exported, referenced, and independently checked later?
- Can a user prove that a public report is the same report produced by the tool at a specific time?

Traditional PDFs often fail these questions. They can be edited, reposted, summarized incorrectly, or detached from the source that produced them. Even when the underlying analysis is honest, the delivery format weakens trust. Developers need a report, but ecosystems need provenance.

L2 gas cost is opaque and increasingly DA-dominated

Layer-2 execution changes the gas optimization problem. On Ethereum L1, developers often reason about gas as a single execution-cost number. On modern L2s, the cost users experience is shaped by two components: execution on the L2 and data availability or L1 settlement cost. The relative weight of those components changes with calldata shape, compression, batching, L1 fee conditions, and chain-specific fee accounting.

Mantle is designed for high-throughput, low-cost execution, but developers still need chain-aware cost analysis. A change that looks small in EVM execution gas may matter if it reduces calldata or bytecode footprint. Conversely, a micro-optimization may be irrelevant if the saving is dwarfed by DA cost or if the function is rarely called. Generic gas advice is not enough. Builders need optimization reports that separate:

- L2 execution gas saved per call,
- data or deployment footprint changes,
- annualized savings assumptions,
- measured versus estimated deltas,
- source of pricing assumptions,
- and whether a patch still compiles after the suggested change.

Without this separation, gas tooling becomes a credibility problem. It may claim savings that are real on one chain, approximate on another, or meaningless when applied to production traffic.

Developer workflows need proof, not just advice

undefined

Security and cost reports become more valuable when they can be embedded into workflows. A founder wants a public proof for a demo or partner review. A protocol team wants a CI comment on every pull request. A developer relations team wants an ecosystem leaderboard of concrete gas reductions. A reviewer wants a durable link to a report that can be checked later.

Archon's position is that audit and gas tooling should produce structured, composable artifacts:

- human-readable reports for teams,
- machine-readable JSON for integrations,
- CI comments for pull requests,
- public leaderboard rows for traction and comparison,
- and on-chain proof references for reputation and verification.

The PDF should not disappear. It should become one export from a stronger underlying evidence system.

Thesis: verifiable DevTools

Archon's thesis is simple: DevTools for crypto should be verifiable by default.

A verifiable DevTool does not ask users to trust a screenshot, marketing claim, or opaque report. It produces artifacts that can be inspected by humans, consumed by machines, and anchored to public identifiers. It is honest about limits. It separates deterministic analysis from model-assisted explanation. It treats chain-specific economics as part of correctness. It gives developers a practical workflow without pretending that automation replaces expert judgment.

Archon applies this thesis to smart-contract auditing and Mantle gas optimization. The platform is built around four principles.

First, every important artifact should have provenance. Audit reports and gas reports are derived from source inputs, assumptions, hashes, timestamps, and network metadata. The product records those details so an output can be traced back to its input.

Second, explanations should be useful but not magical. AI is valuable for summarizing findings, explaining risk, and generating test scaffolds, but the system should keep deterministic categories, line references, source snippets, and rule provenance visible. A model can improve developer understanding; it should not erase the audit trail.

Third, gas optimization must be chain-aware. Mantle optimization is not only about reducing opcodes. It is about understanding the cost split between L2 execution and DA/L1 components, then presenting the economic impact under explicit assumptions.

Fourth, trust should be portable. A report should be linkable, exportable, and anchorable. ERC-8004-style identity and on-chain proof records give Archon a path toward reputation that is not trapped inside the application database.

The goal is not to create a perfect oracle of security. The goal is to make automated security and cost analysis more transparent, more reproducible, and more useful in the places where developers actually work.

System architecture

Archon is organized as a product system rather than a single analyzer. The web application, worker pipeline, database, queue, AI enrichment layer, Mantle RPC access, proof registry integration, GitHub Action, docs site, public leaderboard, and API reference all serve the same output model: produce a useful artifact and preserve enough context to verify it.

undefined

[Architecture diagram: Archon system architecture]

At a high level, the platform contains seven layers.

- Input layer. Users submit Solidity through paste, upload, GitHub import, sample contracts, or verified Mantle address lookup. Inputs are normalized, validated, size-limited, and hashed.
- Queue and worker layer. Audit and gas jobs are persisted and executed asynchronously so the UI and API can show progress, logs, and failures without blocking the request lifecycle.
- Analysis layer. The audit engine combines compilation, static analysis, Mantle-specific deterministic rules, gas-opportunity detection, and optional AI enrichment.
- Gas measurement layer. The gas optimizer compiles source, detects optimization candidates, estimates or measures L2 execution deltas, models DA/L1 effects from receipt-calibrated data, and records assumptions.
- Artifact layer. Reports, findings, optimizations, tests, proof metadata, source hashes, and logs are stored in the database and exposed through app pages and API endpoints.
- Verification layer. Report hashes and proof metadata can be prepared and anchored through Archon's on-chain proof flow. Public proof pages and metadata endpoints make verification possible outside the UI.
- Distribution layer. Fumadocs documentation, the public gas leaderboard, the Scalar API reference, and the GitHub Action turn Archon from a dashboard into a developer platform.

This architecture is deliberately modular. Static analysis can improve without changing the proof model. Gas pricing can be refined without rewriting audit reports. The proof registry can evolve without forcing the product to abandon local report exports. The system is useful today while leaving room for stronger verification over time.

The Audit Engine

Archon's Audit Engine is designed for fast, explainable security feedback on Mantle-facing Solidity contracts. It is not a black-box model that emits generic warnings. It is a pipeline of deterministic and assisted stages, each with a role in the final report.

The pipeline begins with source normalization. Archon validates that the input contains Solidity, identifies a contract name, detects the pragma, writes a temporary compilation workspace, and records basic scope metadata. This stage matters because the credibility of the report depends on knowing what was actually analyzed.

The next stage is compilation. Compilation is both a syntax gate and a source-of-truth check for bytecode-dependent analysis. If the source cannot compile under a compatible compiler, Archon should fail clearly rather than invent findings. A compile failure is not a product failure; it is a signal that the source, imports, compiler version, or environment needs attention. For exact pragmas such as pragma solidity 0.8.24;, the compiler must match the requested version exactly. For range pragmas such as ^0.8.24, newer compatible compilers can be used.

After compilation, Archon runs static analysis. Static analyzers are valuable because they encode known vulnerability patterns, compiler observations, and optimization hints. Their output is mapped into Archon's report model: severity, category, title, file, line range, snippet, confidence, source, and remediation guidance.

Archon also runs Mantle-specific rule checks. These rules focus on patterns that are especially relevant to applications deployed or integrated on Mantle, including authorization assumptions, timestamp-sensitive settlement, oracle freshness, reentrancy around value transfers, slippage enforcement, and storage iteration that can become operationally expensive. These checks are deterministic and designed to be explainable rather than theatrical.

AI enrichment is applied as a communication layer. The goal is to turn raw findings into developer-friendly summaries, exploit scenarios, and recommended fixes while preserving the underlying deterministic evidence. The model is not treated as an unquestioned authority. Findings remain tied to source snippets and rule/static-analysis provenance.

undefined

The output is an audit report with a risk score, severity counts, finding list, executive summary, source scope, and exportable metadata. Reports can be linked, rendered in the app, exported through API endpoints, and used as the basis for proof metadata.

A good automated audit report should help a developer decide what to fix next. A great one should also make it clear where the evidence came from, what the system did not prove, and how to reproduce the analysis path.

The Mantle Gas Optimization Engine

Archon's Gas Optimization Engine is built around a specific belief: L2 gas optimization must explain both execution cost and data cost. Mantle developers need to know whether an optimization reduces L2 execution gas, DA/L1 cost, deployment footprint, or only theoretical complexity.

The engine begins with the same input discipline as the audit pipeline. Source is validated, hashed, compiled, and assigned a report record. The optimizer then applies a catalog of Solidity gas rules. Examples include redundant initialization, storage-read caching, unchecked arithmetic in safe contexts, calldata usage, visibility choices, event data shape, and bytecode-size-sensitive patterns. Each opportunity is recorded with a rule id, title, category, file, location, before/after text, safety note, confidence, patch data, and pricing labels.

Archon separates three types of gas evidence:

- Measured. A patch or scenario was compiled and measured through a harness or gas-report path.
- Estimated. The rule has a deterministic estimate, but no full measurement was produced for that exact contract shape.
- Unpriced. The rule may improve code quality or theoretical gas behavior, but Archon does not claim a quantified saving.

This distinction matters. A product that labels every suggestion as a precise saving will mislead users. Archon should be useful even when it says, “this is an estimate,” or “this is unpriced.”

DA-fee model

Mantle transaction cost can be reasoned about as a combination of L2 execution and DA/L1 data cost:

total_cost_per_call "H (l2_gas_used × l2_gas_price) + l1_or_da_fee

For optimization reporting, the saving is the difference between the baseline and optimized paths:

saving_per_call "H (9FÃ%öv 0 × l2_gas_price) + 9FÃ ö÷%öF ö`ee annual_saving "H saving_per_call × calls_per_year × MNT_USD

Archon treats this as an explicit assumption model, not a hidden constant. Reports expose call-volume assumptions, MNT/USD assumption, L2 gas price source, DA model mode, calldata byte profile, and whether the DA estimate is receipt-calibrated or degraded.

The DA component is especially important for deployment and calldata-heavy flows. Bytecode size and calldata shape can change cost even when opcode-level execution appears similar. Zero bytes and non-zero bytes have different calldata gas profiles, and L2 systems may compress or account for data differently. Archon's current model uses deterministic calldata profiling plus receipt-calibrated validation data where available. It avoids presenting stale or chain-inaccurate formulas as verified truth.

Worked L1/L2 example

Consider a frequently called function on Mantle. Suppose Archon identifies a safe optimization that reduces L2 execution by 1,200 gas per call and reduces calldata/DA-equivalent cost by 8,000 wei per call under the active model. Suppose the current L2 gas price is 30 gwei, the function is called 250,000 times per year, and the reporting assumption is 1 MNT = 1 USD.

undefined

The L2 execution saving is:

1,200 gas × 30 gwei = 36,000 gwei = 0.000036 MNT per call

The DA/L1 saving is:

8,000 wei = 0.000000000000008 MNT per call

The combined per-call saving is approximately:

0.000036000000008 MNT

Annualized across 250,000 calls:

0.000036000000008 × 250,000 "H 9.000000002 MNT/year

At 1 USD per MNT, that is roughly 9.00 USD/year. If the function is called 25 million times per year, the same per-call improvement becomes roughly 900 USD/year. The optimization did not change; the business relevance changed because traffic changed. This is why Archon reports assumptions instead of pretending there is one universal savings number.

A second example shows why DA awareness matters. A deployment optimization that reduces bytecode by 2,000 bytes may have little effect on a single runtime function but can materially reduce deployment data cost and make contracts easier to fit within operational constraints. Archon records deployment-footprint effects separately from per-call L2 execution savings so teams can reason about both.

ERC-8004 identity & on-chain proofs

Archon uses an on-chain identity and proof model to make reports referenceable beyond the application UI. The long-term direction aligns with ERC-8004's agent identity and reputation framing: agents should be discoverable, attributable, and able to accumulate reputation through externally verifiable work.

In Archon's current model, the platform has an agent identity reference and a proof registry path. A report can produce proof metadata that includes report scope, findings, risk score, hashes, timestamps, and network context. That metadata can be prepared for self-custody verification or logged through the configured proof flow. Public proof pages and API endpoints expose the result.

The important design choice is that proofs do not claim more than they prove. A proof can establish that a specific report artifact or metadata package was produced and referenced at a point in time. It does not prove that every finding is complete, that no vulnerability exists, or that a human auditor has endorsed the code. The proof is provenance, not omniscience.

This distinction is essential for honest security tooling. On-chain proof should strengthen the evidence trail, not become a marketing trick. The right claim is: “this report, with this hash and metadata, was produced by this system and can be checked.” The wrong claim is: “this contract is safe because a proof exists.”

As the ecosystem matures, Archon can attach more signals to identity: CI run history, accepted fixes, leaderboard records, proof verification status, and reputation events. The result is an audit agent whose work becomes more accountable over time.

Public validation and challenge ledger

Verification should not be a one-way announcement. A public report becomes more trustworthy when readers can attach disagreement, evidence, and alternative interpretation to a specific finding or optimization. Archon therefore includes a scoped challenge ledger.

The current challenge flow is intentionally conservative. Anyone can submit a challenge against an audit report, finding, gas report, or optimization. Archon records the challenge in Supabase with target identifiers, title, rationale, optional evidence URL, status, timestamp, and a deterministic challenge hash. If the challenged artifact already has an ArchonProofRegistry transaction, report hash, or gas anchor, the challenge stores that reference as context.

undefined

This design extends the trustless thesis without introducing a deadline-risky new contract. A challenge does not rewrite the original report. It does not claim that the challenger is correct. It creates a durable, public, hash-addressable record of dispute that can be reviewed alongside the report. That is the right level of verification for the current product: transparent, scoped, and honest about what has and has not been proven.

A future version could anchor challenge hashes directly through an extension or registry event, but that should be designed deliberately and only after the existing proof flow remains stable. Under the current milestone, the correct tradeoff is DB-backed public validation with references to existing proof artifacts.

Trust & verifiability model

Archon's trust model has three layers: deterministic evidence, assisted interpretation, and public anchoring.

Deterministic evidence includes source hashes, compiler settings, static-analysis output, rule ids, line mappings, gas deltas, report hashes, and stored metadata. These are the parts of the system that should be reproducible or at least inspectable.

Assisted interpretation includes AI-generated summaries, exploit narratives, recommended fixes, and test scaffolds. These are useful, but they must remain subordinate to evidence. Archon should make AI output easier to challenge, not harder. If a summary is wrong, the source snippet and finding category should still guide the developer back to the underlying issue.

Public anchoring includes proof metadata, transaction hashes, identity references, public report links, leaderboard rows, and API-accessible artifacts. Anchoring extends the artifact beyond the private session. It allows partners, judges, developers, and ecosystems to evaluate work without relying on screenshots.

The model is designed around transparent limits:

- Static analysis is incomplete and may miss logic bugs.
- AI explanations can be wrong and should be reviewed.
- Gas estimates depend on assumptions and contract shape.
- DA/L1 fee modeling changes as chains evolve.
- Proofs establish provenance, not absolute safety.
- CI comments are decision support, not final review authority.

This honesty is a product feature. Security tools that overclaim create false confidence. Archon should earn trust by saying exactly what it did, what it found, what it measured, and what remains outside scope.

Business model & GTM

Archon is positioned as verifiable security and cost infrastructure for Mantle builders, with a go-to-market path that starts with developers and expands into teams, protocols, and ecosystem programs.

The freemium product gives individual builders a fast path to scan contracts, understand findings, optimize gas, and generate proof artifacts. This creates adoption because the first useful result does not require procurement or a sales call.

The CI product turns Archon into workflow infrastructure. The archon-gas-action GitHub Action lets teams run gas optimization on pull requests, receive Mantle-aware gas-diff comments, and optionally fail builds when regressions exceed a configured threshold. This is the natural path from “I tried the tool” to “my team depends on the tool.”

The leaderboard creates public proof of usefulness. A gas leaderboard built from completed reports lets Archon show real optimization outcomes, label samples clearly, and link back to report artifacts. This helps developer relations, hackathon judging, and ecosystem growth because traction is visible in product data rather than private dashboards.

undefined

The API product gives power users and partners direct access to scans, reports, proofs, gas reports, and leaderboard data. The OpenAPI/Scalar reference makes integration easier and establishes Archon as a platform rather than only an app.

On-chain reputation is the long-term wedge. As reports, proofs, CI runs, and accepted improvements accumulate, Archon can become a reputation-bearing agent in the Mantle ecosystem. Protocols may eventually evaluate not only a single report, but the history of an agent's verified outputs.

Commercially, this supports several tiers:

- free individual scans and public examples,
- paid higher-volume scans and CI usage,
- team plans for private reports and collaboration,
- API access for partners and dashboards,
- ecosystem programs for sponsored audits, gas optimization campaigns, and public reputation surfaces.

The GTM motion is therefore product-led first, ecosystem-led second, and enterprise-capable later. Archon should win developer trust before selling compliance language.

Roadmap

Archon's roadmap is organized around increasing verification strength, workflow reach, and analysis depth.

Near term, the priority is reliability. The audit and gas pipelines should handle common Solidity pragmas, imports, uploaded files, GitHub sources, and address-based verified source without surprising users. Error states should be clear. Docs should match product behavior. Public pages should avoid fake claims.

The next product layer is workflow adoption. The GitHub Action should become easy to install, configurable by path and threshold, and safe under GitHub token and rate-limit constraints. Reports should be easy to share. API examples should remain current.

The next analysis layer is deeper measurement. Gas reports should improve harness generation, patch application, and per-rule measurement. Audit reports should improve source mapping, protocol integrations, generated tests, and vulnerability-specific explanations.

The next verification layer is stronger proof UX. Proof pages should make the distinction between prepared metadata, logged proofs, verified self-custody transactions, and on-chain identity clearer. External verifiers should be able to consume metadata without using the app.

The long-term roadmap is an agent reputation network: Archon reports, gas improvements, CI outcomes, proofs, and accepted fixes become a public history of useful security work. The product can then support team dashboards, marketplace-style reputation, ecosystem campaigns, and deeper integration with Mantle developer infrastructure.

Security, limitations & disclaimers

Archon is a developer security tool, not a guarantee of safety. Automated scans can miss vulnerabilities, misunderstand business logic, or overstate impact. AI-generated explanations should be reviewed by humans. Gas recommendations should be tested against the target codebase and deployment process. Proof records should be interpreted as provenance records, not certification of correctness.

Archon is read-only by default. It analyzes source, produces reports, suggests patches, and prepares proof metadata. Users remain responsible for applying fixes, reviewing changes, running test suites, and deciding whether code is production-ready.

undefined

The platform should never encourage blind deployment. The correct use of Archon is to shorten feedback loops, improve evidence quality, make reports easier to verify, and help teams prioritize work. For high-value protocols, Archon should complement expert review, formal methods, fuzzing, invariant tests, economic analysis, operational security review, and incident response planning.

Fee models are also bounded. Mantle gas conditions, DA pricing, compression behavior, and RPC data can change. Archon exposes assumptions because cost estimates should be treated as decision support, not immutable truth.

Finally, public reputation systems must be designed carefully. A leaderboard or proof registry can create incentives to optimize for visible metrics. Archon's product direction should favor truthful labeling, real links, clear sample markers, and transparent limitations over inflated claims.

References

- ERC-8004: Trustless Agents (https://eips.ethereum.org/EIPS/eip-8004)
- Mantle documentation (https://docs.mantle.xyz)
- Solidity documentation (https://docs.soliditylang.org)
- Slither static analyzer (https://github.com/crytic/slither)
- Foundry (https://book.getfoundry.sh)
- Archon API reference (/api-reference)
- Archon gas leaderboard (/gas-leaderboard)
- Archon architecture (/docs/resources/architecture)

undefined